Door Face Panels

Smart Door Anomaly Detection & Cybersecurity for Elderly Care

CSIS 4495-003 – Applied Research Project

Team Members:

Advitiya Sharda [300395470]

Eric Sanjo [300395898]

Reubin Chatta [300394193]

Team Lead: Advitiya Sharda

Section: 003
Date: February 23 , 2026

GitHub Repo:
https://github.com/advitiyasharda/W26_4495_S3_AdvitiyaS

Video: https://youtu.be/S1aYgl6lSdc

Riipen Partner: Door Face Panels (Coquitlam, BC)
Contact: Armin Ghauforian, Founder and CEO
Project Focus: IoT-based smart door security system combining anomaly detection and cybersecurity for elderly care and safety monitoring

1. Introduction

1.1 Domain Overview and Context

Elderly care facilities and assisted living environments face growing challenges around access control, caregiver accountability, and resident safety. A door event — someone entering or leaving — can carry both a safety signal (prolonged inactivity may indicate a fall or health emergency; late-night exits may indicate wandering) and a security signal (unknown person at the door; repeated failed access attempts). Door Face Panels, based in Coquitlam, BC, is building modular smart door systems for this population. Our project directly supports that product vision by implementing an edge-first smart door security platform: local facial recognition, event logging, rules-based threat alerting, and a modern caregiver-facing dashboard.

1.2 Problem Framing and Research Questions

Our work addresses four key challenges:

- **RQ1:** How can facial recognition access control be combined with a real-time monitoring dashboard to give caregivers clear, actionable visibility into door events?
- **RQ2:** What rule-based threat checks and ML anomaly detection can reliably identify both security anomalies (unauthorized access, repeated failures) and behavioral anomalies (inactivity, wandering, unusual patterns)?
- **RQ3:** How can the system support privacy and compliance requirements (PIPEDA/GDPR mindset) by storing only necessary data and maintaining a full audit trail of system actions?
- **RQ4:** What deployment strategies allow efficient face recognition and anomaly detection on resource-constrained edge hardware (Raspberry Pi / Jetson)?

1.3 Literature Review and Knowledge Gaps

Prior work supports the practicality of OpenCV-based face detection and lightweight feature extraction for edge systems (Bradski & Kaehler, 2008). Isolation Forest is well established as a baseline for unsupervised anomaly detection on tabular and time-series style features (Liu et al., 2008). IoT security literature supports lightweight rule-based detection on constrained devices. However, a gap exists in integrating behavioral anomaly detection, security threat detection, and a caregiver-facing compliance dashboard into a single, edge-deployable, privacy-conscious system for elderly care. Our work addresses this by combining all three layers — recognition, threat rules, and anomaly detection — with a modern web dashboard and a complete audit trail.

1.4 Initial Hypotheses, Assumptions, and Benefits

Assumptions:

- Door activity patterns are a practical proxy for both security incidents and resident well-being.

- A combination of rule-based alerts and simple ML anomaly detection provides practical, explainable value without cloud infrastructure.
- Local processing and local storage reduces privacy risk and latency, and is viable for a course-term prototype.

Benefits:

- Better caregiver awareness and accountability through real-time dashboards.
- Proactive safety monitoring via inactivity and behavioral anomaly detection.
- Privacy-preserving architecture (edge-only, no cloud uploads).
- Compliance support through a full audit trail.
- Strong product foundation for Door Face Panels.

2. Summary of the Initially Proposed Research Project

The original proposal (January 26, 2026) described an IoT smart door security system combining facial recognition, ML anomaly detection, rules-based threat alerting, and a caregiver dashboard, deployed locally on Raspberry Pi / Jetson hardware without cloud dependency. The six core objectives were:

- Facial Recognition & Access Control: identify registered users at the door with <500ms inference latency and log every entry/exit.
- Anomaly Detection (ML): train Isolation Forest (and optionally an LSTM autoencoder) on synthetic access patterns to detect unusual behavioral signals.
- Threat Detection (Rules + ML): flag suspicious access patterns — repeated failures, unusual times, frequency spikes, inactivity.
- Caregiver Dashboard: a web interface showing logs, alerts, analytics, and a compliance/audit trail.
- Security & Compliance: local encrypted storage, audit logging, PIPEDA-aligned data handling.
- Edge Deployment Strategy: document how models run on Raspberry Pi/Jetson without cloud dependency.

Roles were assigned as: Advitiya — cybersecurity, facial recognition pipeline, threat rules; Eric — synthetic dataset generation and anomaly detection; Reubin — anomaly models, dashboard build, and system integration.

3. Changes to the Proposal

Change 1: Dashboard Technology — from Flask HTML/CSS to Next.js (React + Tailwind CSS)

What changed: We built and validated an initial HTML/CSS/JS prototype (kept in the repo as 'doortest' for reference), then migrated the full dashboard to a Next.js App Router application with Tailwind CSS and React components.

Justification: Next.js provides a cleaner component model for charts, tables, and navigation; built-in API proxy rewrites eliminate CORS friction between the frontend and Flask; and the approach is easier to maintain and extend (additional pages, filtering, CSV export) than raw HTML templates. The migration from prototype to Next.js happened during the two weeks of this midterm reporting period.

Change 2: Face Recognition — OpenCV-first baseline (TensorFlow Lite as future goal)

What changed: The proposal listed TensorFlow Lite as a face recognition option. The implemented working baseline uses OpenCV (Haar Cascade detection + HOG-based feature extraction), with optional support for the 'face_recognition' (dlib) library when available.

Justification: OpenCV is reliably cross-platform (macOS/Windows/Linux), requires no additional model downloads, and is straightforward to set up on development machines and later on Raspberry Pi. TensorFlow Lite remains a future upgrade path for improved accuracy and Pi optimization.

Change 3: Working end-to-end integration prioritized over ML depth at midterm

What changed: We prioritized building a fully connected end-to-end pipeline (camera frame → recognition → log → threat alert → dashboard display) by midterm rather than maximizing the complexity of the anomaly detection model.

Justification: A working, demoable product by midterm is explicitly required by the course. The Isolation Forest module exists in the codebase (trained on synthetic data) and is ready for deeper pipeline integration in the second half of the term. Rule-based alerts provide immediate, explainable value in the meantime.

Change 4: Backend port standardized to 5001 (not default 5000)

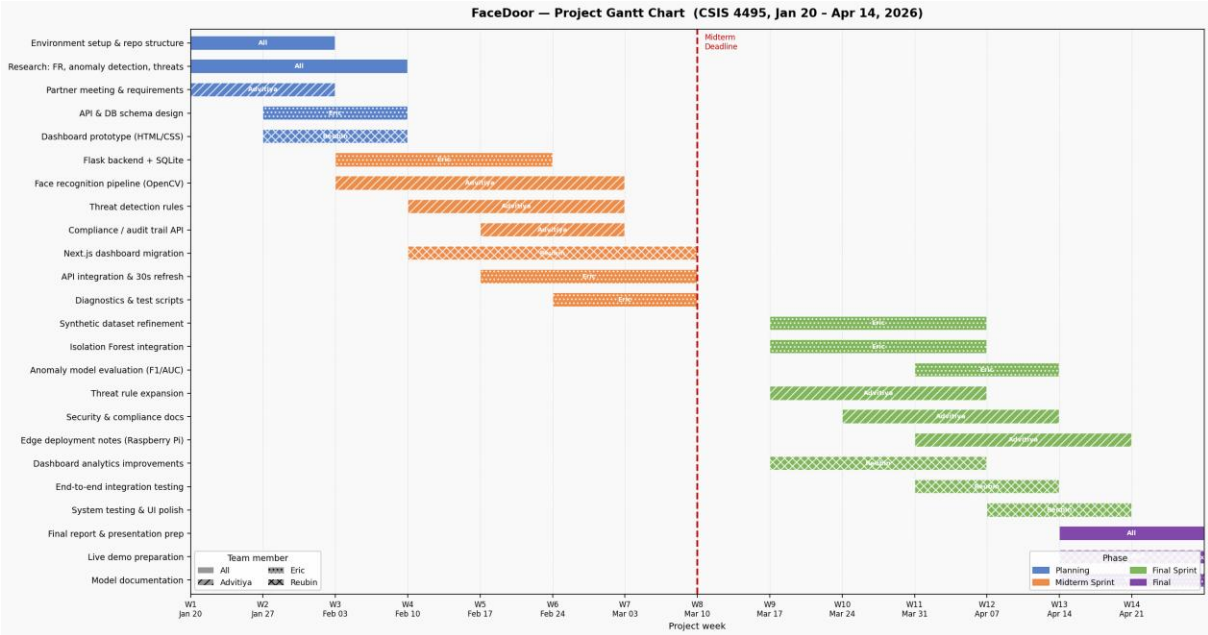
What changed: Flask runs on port 5001 by default for this project; the Next.js proxy target in next.config.ts is configured to match.

Justification: Standardizing on a non-default port reduces conflicts with other local services and makes the setup explicit and documented in the README and installation instructions.

4. Project Planning and Timeline

4.1 Completed Milestones (up to midterm)

- Phase 1 (Jan 20–Feb 2): Environment setup, repo structure, research, API design.
- Phase 2 partial (Feb 3–Feb 23): Core implementation — recognition pipeline, access logging, threat detection, audit trail, Next.js dashboard, diagnostics, testing scripts.



4.2 Remaining Timeline (Feb 24 – end of term)

Phase / Week	Advitiya	Eric	Reubin
W8–9 (Feb 24–Mar 8)	Recognition accuracy tuning; security hardening notes	Synthetic dataset refinement; Isolation Forest integration	Dashboard analytics improvements; anomaly score display
W10–11 (Mar 9–Mar 22)	Threat rules expansion; compliance documentation	Anomaly model evaluation (precision/recall/F1)	End-to-end integration testing; UI polish
W12 (Mar 23–Mar 29)	Performance metrics + edge deployment notes	Dataset validation documentation	System testing; final documentation review
W13–14 (Apr 1–Apr 14)	Final report lead; presentation planning	Model documentation + demo support	Live demo preparation; repo final clean-up

4.3 Team Responsibilities (remainder of term)

Member	Primary Responsibilities
Advitiya Sharda (Team Lead)	Facial recognition pipeline; threat detection and security logic; compliance/audit documentation; Riipen partner communication; joint submission lead.
Eric Sanjo	Synthetic dataset generation and validation; Isolation Forest training and evaluation; anomaly detection pipeline integration; documentation of data methodology.

Reubin Chatta	Dashboard development and integration; anomaly score visualization; system integration and end-to-end testing; deployment documentation.

5. Implemented Features (Midterm Deliverables)

The two weeks of this reporting period (February 10–23, 2026) were the core implementation sprint, building directly on the planning and exploratory work from Progress Report 1. Below we describe each implemented feature with technical detail and individual attribution. Screenshots of the running application and relevant code should be inserted at the markers indicated.

5.1 Backend API and Database (Flask + SQLite)

Feature description

We implemented a Flask REST API backed by SQLite that serves as the single data layer for the entire system. The application factory (api/__init__.py) creates the Flask app, enables CORS for the Next.js dev server, initializes the database, loads face encodings from data/samples/ at startup, and registers API blueprints.

Individual contributions

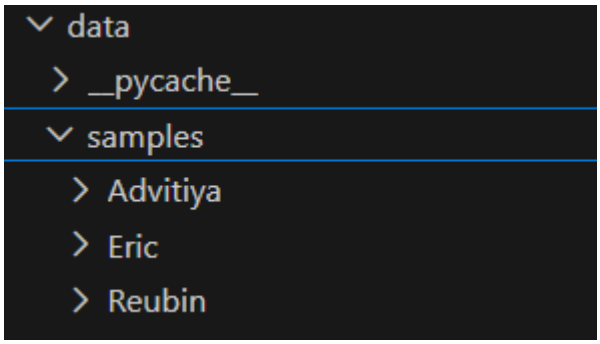
- Advitiya: database schema design; audit log fields; security-aligned logging approach.
- Eric: API endpoint integration; data flow from recognition to storage; access log structure.
- Reubin: integration planning; ensuring dashboard endpoints match UI needs.

[INSERT SCREENSHOT: terminal showing backend startup and API response example]

5.2 Facial Recognition Pipeline (OpenCV, HOG Features)

Feature description

We implemented a FacialRecognitionEngine class (api/facial_recognition.py) that handles face detection and identity matching. At application startup, the engine loads sample images from data/samples/<PersonName>/, detects faces using OpenCV Haar Cascade, extracts a 128-dimensional feature vector per face using HOG-based gradients and histograms, and registers those encodings in memory keyed by person name.



Recognition flow

- A frame (base64-encoded image) arrives at /api/facial_recognition.
- OpenCV detects face bounding boxes in the frame.

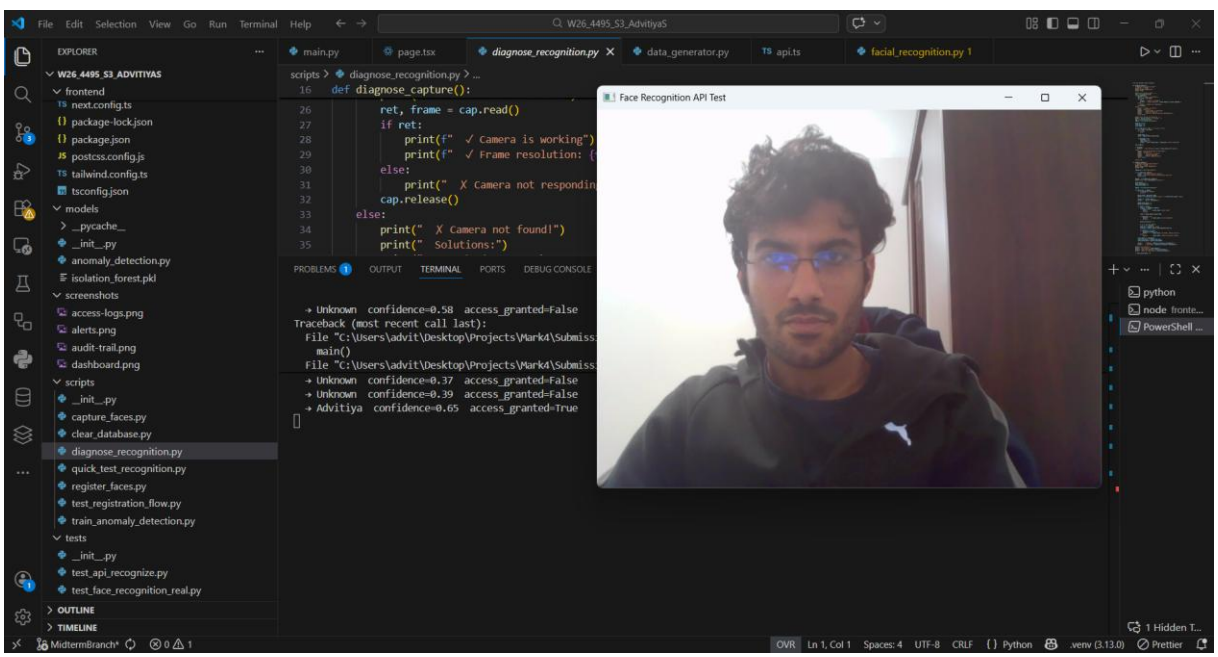
- For each detected face, a 128-dim HOG feature vector is extracted.
- The vector is compared to all stored encodings by Euclidean distance.
- The best match below a distance threshold — and with sufficient margin over the second-best from a different person — is accepted as a recognition result.
- The result (person name + confidence, or Unknown) is returned and stored in access_logs.

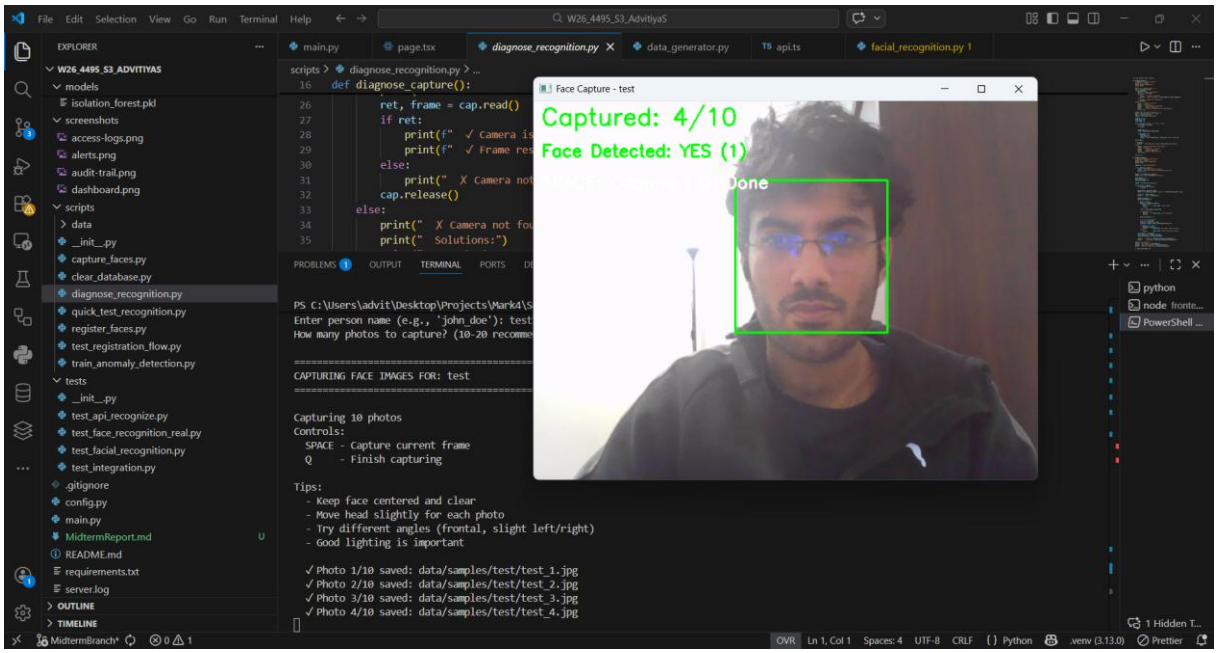
Tuning and diagnostics

We implemented a diagnostics script (scripts/diagnose_recognition.py) that checks sample quality and flags suspicious cross-user similarity. We also built a live webcam test (tests/test_face_recognition_real.py) and fixed its recognition-rate metric to report 'frames with recognized face / frames with detected face' (always 0–100%) rather than a confusing ratio exceeding 100%.

Individual contributions

- Advitiya: facial recognition pipeline design and implementation; threshold tuning; audit/compliance logging of recognition results.
- Eric: integration of recognition results into the API response; testing scripts and diagnostics.
- Reubin: sample registration support; face capture script (scripts/capture_faces.py).





5.4 Next.js Caregiver Dashboard (UI migration + components)

Feature description

We migrated the caregiver dashboard from an early HTML/CSS/JS prototype (preserved in 'dashboard/' and 'doortest/' folders for reference) to a modern Next.js 16 application with React 19, Tailwind CSS, and Recharts for data visualization.

Dashboard pages

Page (URL)	Content
Dashboard (/)	KPI stat cards (entries, denials, threats); hourly bar chart; access donut chart; recent logs table
Alerts (/alerts)	Threat alert cards filtered by ALL / HIGH / CRITICAL severity
Access Logs (/logs)	Full paginated access log table; Registered People panel with delete-user feature
Audit Trail (/compliance)	Compliance audit log with actor/resource/result; CSV export

Reusable components

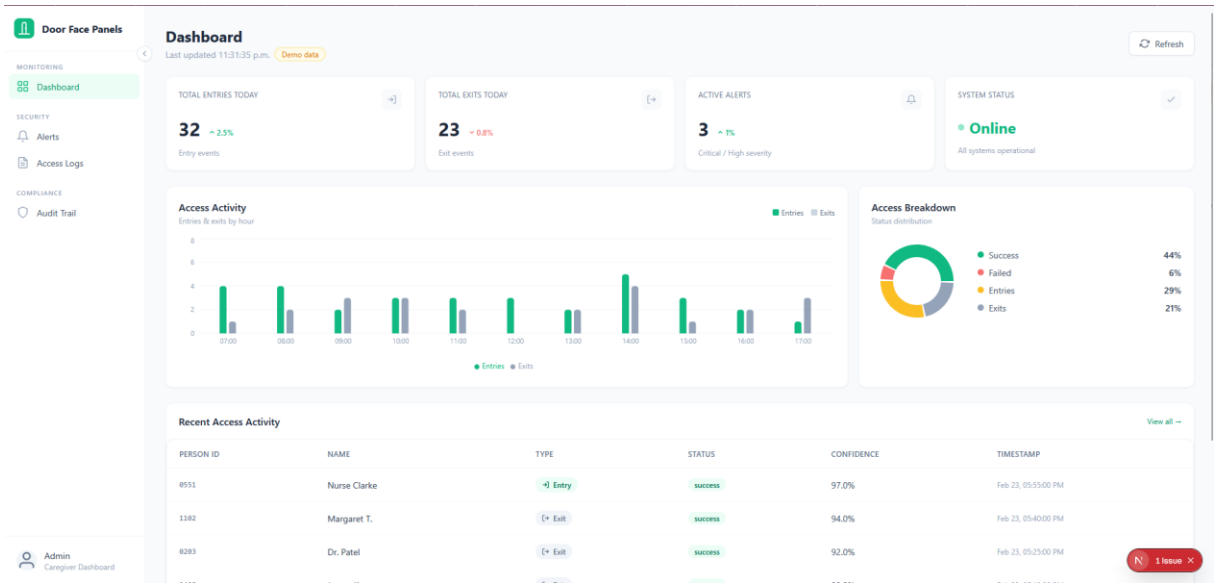
- Sidebar: collapsible navigation sidebar shared across all pages.
- StatCard: KPI card with value, label, trend indicator, and icon.
- AccessChart (Recharts): hourly bar chart of entries and exits.
- StatusDonut (Recharts): donut chart of access outcomes (granted / denied / unknown).
- AccessLogsTable: paginated table with Entry/Exit and Status badges.
- AlertList: threat card list with severity badges and filtering.
- AuditTable: compliance log with CSV export.
- StatusBadge: small coloured pill component for status display.

API integration and auto-refresh

All frontend data comes from the Flask API through a Next.js proxy rewrite (next.config.ts). The dashboard calls /api/stats and /api/logs on load and then refreshes every 30 seconds automatically, so the view stays current without manual refresh.

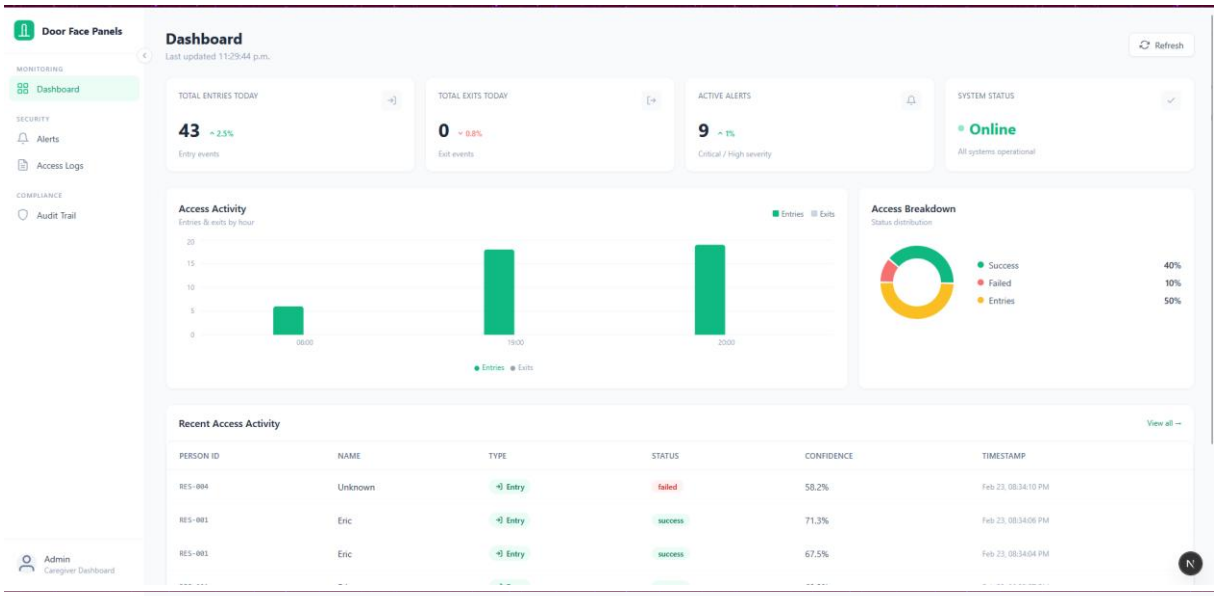
Demo mode

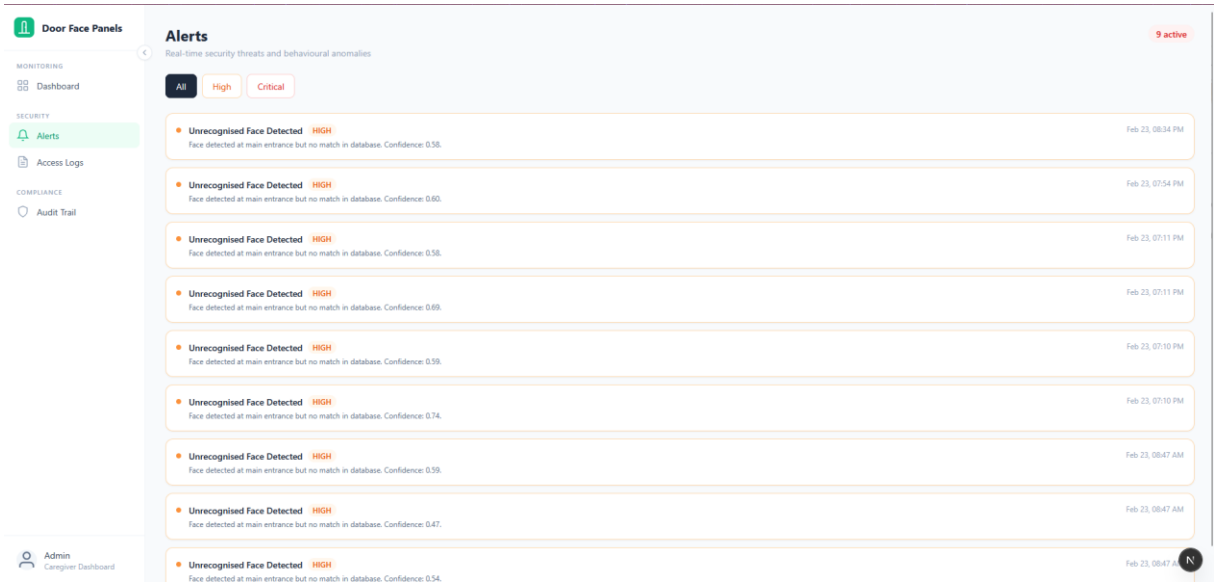
When the backend is unavailable or the database is empty, the frontend automatically shows realistic seeded demo data with a visible 'Demo data' badge so the UI remains presentable in any state.



Individual contributions

- Reubin: dashboard planning; UI migration story; component design and layout.
- Eric: API client (lib/api.ts); proxy configuration; refresh loop; integration wiring.
- Advitiya: compliance page framing; audit trail requirements; Registered People management.


Door Face Panels
MONITORING
Dashboard
SECURITY
Alerts
Access Logs
COMPLIANCE
Audit Trail
Admin
Caregiver Dashboard
Dashboard
Last updated 11:29:44 p.m.
Refresh
TOTAL ENTRIES TODAY
43 ^ 2.5%
Entry events
TOTAL EXITS TODAY
0 ~ 0.8%
Exit events
ACTIVE ALERTS
9 ^ 1%
Critical / High severity
SYSTEM STATUS
Online
All systems operational
Access Activity
Entries & exits by hour
Entries
Exits
20
15
10
5
0
0600
1900
2000
Entries
Exits
Access Breakdown
Status distribution
Success 40%
Failed 10%
Entries 50%
Recent Access Activity
View all →
PERSON ID
NAME
TYPE
STATUS
CONFIDENCE
TIMESTAMP
RES-004 Unknown Entry failed 58.2% Feb 23, 08:34:10 PM
RES-001 Eric Entry success 71.3% Feb 23, 08:34:06 PM
RES-001 Eric Entry success 67.5% Feb 23, 08:34:04 PM


Door Face Panels
MONITORING
Dashboard
SECURITY
Alerts
Access Logs
COMPLIANCE
Audit Trail
Admin
Caregiver Dashboard
Alerts
Real-time security threats and behavioural anomalies
9 active
All
High
Critical
Unrecognised Face Detected HIGH
Face detected at main entrance but no match in database. Confidence: 0.58.
Feb 23, 08:34 PM
Unrecognised Face Detected HIGH
Face detected at main entrance but no match in database. Confidence: 0.60.
Feb 23, 07:54 PM
Unrecognised Face Detected HIGH
Face detected at main entrance but no match in database. Confidence: 0.58.
Feb 23, 07:11 PM
Unrecognised Face Detected HIGH
Face detected at main entrance but no match in database. Confidence: 0.69.
Feb 23, 07:11 PM
Unrecognised Face Detected HIGH
Face detected at main entrance but no match in database. Confidence: 0.59.
Feb 23, 07:10 PM
Unrecognised Face Detected HIGH
Face detected at main entrance but no match in database. Confidence: 0.74.
Feb 23, 07:10 PM
Unrecognised Face Detected HIGH
Face detected at main entrance but no match in database. Confidence: 0.59.
Feb 23, 08:47 AM
Unrecognised Face Detected HIGH
Face detected at main entrance but no match in database. Confidence: 0.47.
Feb 23, 08:47 AM
Unrecognised Face Detected HIGH
Face detected at main entrance but no match in database. Confidence: 0.54.

Door Face Panels

MONITORING
Dashboard

SECURITY
Alerts
Access Logs

COMPLIANCE
Audit Trail

Admin
Caregiver Dashboard

Access Logs

Full audit trail of all door access events

20 records

Successful 19 | Failed 1 | Registered People 4

PERSON ID	NAME	TYPE	STATUS	CONFIDENCE	TIMESTAMP
RES-004	Unknown	Entry	failed	58.2%	Feb 23, 08:34:10 PM
RES-001	Eric	Entry	success	71.3%	Feb 23, 08:34:06 PM
RES-001	Eric	Entry	success	67.5%	Feb 23, 08:34:04 PM
RES-001	Eric	Entry	success	69.2%	Feb 23, 08:33:57 PM
RES-001	Eric	Entry	success	69.7%	Feb 23, 08:33:53 PM
RES-001	Eric	Entry	success	66.7%	Feb 23, 08:33:50 PM
RES-001	Eric	Entry	success	69.5%	Feb 23, 08:32:40 PM
RES-005	Advitiya	Entry	success	77.5%	Feb 23, 08:32:35 PM
RES-001	Eric	Entry	success	70.8%	Feb 23, 08:32:31 PM
RES-005	Advitiya	Entry	success	75.7%	Feb 23, 08:31:27 PM
RES-001	Eric	Entry	success	64.9%	Feb 23, 08:31:22 PM
RES-001	Eric	Entry	success	66.3%	Feb 23, 08:31:19 PM
RES-001	Eric	Entry	success	75.5%	Feb 23, 08:31:12 PM

Door Face Panels

MONITORING
Dashboard

SECURITY
Alerts
Access Logs

COMPLIANCE
Audit Trail

Admin
Caregiver Dashboard

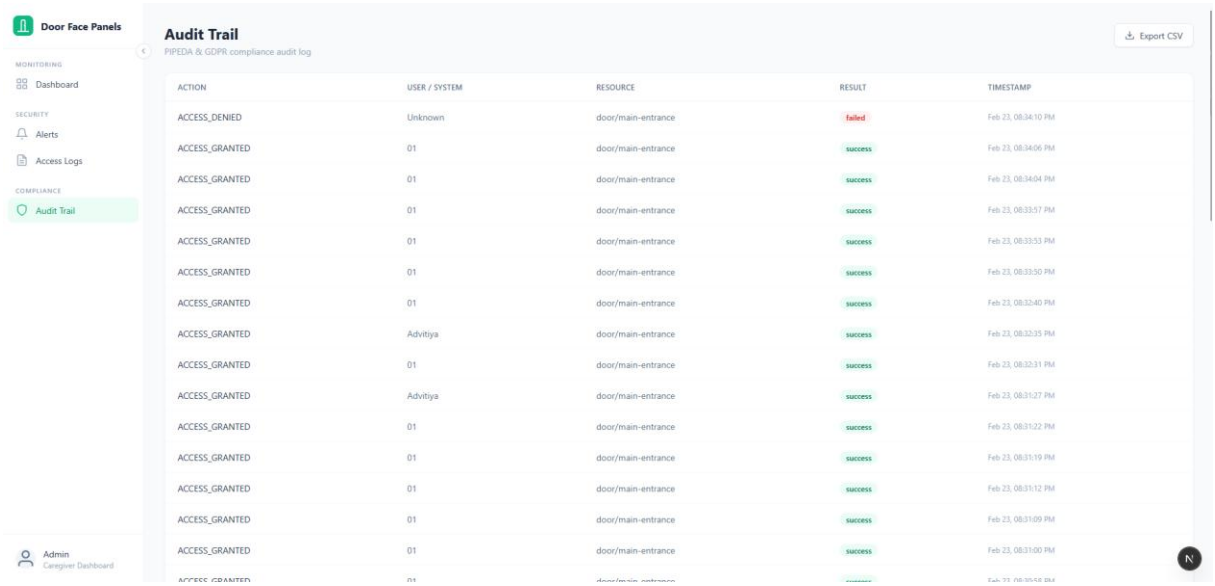
Alerts

Real-time security threats and behavioural anomalies

9 active

All | High | Critical

- **Unrecognised Face Detected** HIGH — Feb 23, 08:34 PM
  Face detected at main entrance but no match in database. Confidence: 0.58.
- **Unrecognised Face Detected** HIGH — Feb 23, 07:54 PM
  Face detected at main entrance but no match in database. Confidence: 0.60.
- **Unrecognised Face Detected** HIGH — Feb 23, 07:11 PM
  Face detected at main entrance but no match in database. Confidence: 0.58.
- **Unrecognised Face Detected** HIGH — Feb 23, 07:11 PM
  Face detected at main entrance but no match in database. Confidence: 0.69.
- **Unrecognised Face Detected** HIGH — Feb 23, 07:10 PM
  Face detected at main entrance but no match in database. Confidence: 0.59.
- **Unrecognised Face Detected** HIGH — Feb 23, 07:10 PM
  Face detected at main entrance but no match in database. Confidence: 0.74.
- **Unrecognised Face Detected** HIGH — Feb 23, 08:47 AM
  Face detected at main entrance but no match in database. Confidence: 0.59.
- **Unrecognised Face Detected** HIGH — Feb 23, 08:47 AM
  Face detected at main entrance but no match in database. Confidence: 0.47.
- **Unrecognised Face Detected** HIGH — Feb 23, 08:47 A
  Face detected at main entrance but no match in database. Confidence: 0.54.



5.5 Compliance Audit Trail

Feature description

Every system action (recognition event, user registration, deletion, data export, system startup, etc.) is written to the audit_logs table with: timestamp, actor (user ID / 'system'), action, resource, and result (success / failure). The /api/compliance/audit endpoint joins the users table to resolve actor names for display.

This supports PIPEDA / GDPR-aligned accountability: every decision the system makes is traceable. The Audit Trail page exposes these logs and a one-click CSV export.

Individual contributions

- Advitiya: audit log fields; compliance narrative; PIPEDA alignment rationale.
- Eric: audit query implementation (users JOIN) so names appear in UI rather than raw IDs.
- Reubin: Audit Trail page; CSV export button.

5.6 Diagnostics, Scripts, and Tests

Feature description

We created a full suite of utility scripts (scripts/) and tests (tests/) to validate the system during development and demonstration.

Scripts

Script	Purpose
capture_faces.py	Capture face photos from webcam for a named person
register_faces.py	Register captured photos into DB and encodings
clear_database.py	Reset SQLite DB without deleting sample photos
diagnose_recognition.py	Health check: camera, sample quality, distinct-users check, recognition engine, DB

quick_test_recognition.py	Quick photo + webcam recognition test
train_anomaly_detection.py	Generate synthetic data and train Isolation Forest

Tests

Test	Purpose
test_api_recognize.py	Live webcam frames → /api/recognize → validate responses
test_face_recognition_real.py	Extended webcam recognition with per-frame stats and summary
test_facial_recognition.py	Component-level face detection and feature extraction tests
test_integration.py	End-to-end pipeline integration test

Notable: distinct-user diagnostic check

The diagnose_recognition.py script computes within-person and between-person encoding distances and flags any sample photo that is closer (in feature space) to another person than to its own registered group. This helps identify misplaced or visually ambiguous sample photos that cause 'Unknown' misclassifications at runtime.

Individual contributions

- Eric: test_face_recognition_real.py; diagnostic scripts; quick_test_recognition.py.
- Advitiya: register_faces.py; clear_database.py; data handling and sample management.
- Reubin: capture_faces.py; test_integration.py; system-level test coverage.

6. AI Use Section

6.1 Table of AI Tools and Specific Use

AI Tool	Version / Account	Specific use	Value addition
ChatGPT	GPT-4 / Free student account	Explaining concepts (Isolation Forest, audit logs, HOG features); drafting and rephrasing report text; brainstorming threat rules and test plans	We validated all suggestions against hardware constraints and partner requirements; rewrote in our own words; tuned parameters through actual test runs
Cursor AI (IDE assistant)	Cursor 0.x / Student / Pro trial	Code navigation; implementation iteration; refactoring; troubleshooting; README and documentation updates	We designed the architecture and data model; reviewed all generated code changes; verified behaviour through live runs and tests; ensured compliance with course requirements
GitHub Copilot	Free student tier	Boilerplate code suggestions (routes, templates, loop structures)	We reviewed and corrected all suggestions; security-critical and logic-critical code was written and validated manually

6.2 Appendix: Representative Prompt History

- Prompt 1: "Explain what an audit log should record for a small web app tracking door access events."
- Prompt 2: "Give example threat detection rules for repeated failed access attempts and unusual hours."
- Prompt 3: "How to structure installation instructions for a Flask + Next.js repository with different run ports."
- Prompt 4: "How to compute face match confidence using Euclidean distance and thresholding in a lightweight way."
- Prompt 5: "Draft a 6-minute midterm demo script split across three team members for a door security project."
- Prompt 6: "Explain Isolation Forest in simple terms for a small IoT/door access project."
- Prompt 7: "What are common suspicious patterns in access logs for elderly care?"
- Prompt 8: "Show a simple HOG feature extraction approach for face recognition in OpenCV."
- Prompt 9: "How does Next.js API proxy rewriting work for a frontend-backend split application?"

7. Work Date / Hours Logs

Note: Progress Report 1 covered Jan 27–Feb 9 for Advitiya (~18h), Jan 19–Feb 9 for Eric (~17h), and Jan 19–Feb 9 for Reubin (~17h). This section covers the two-week midterm implementation sprint: February 10–23, 2026.

7.1 Advitiya Sharda (300395470) — Team Lead

Date	Hours	Description of Work Done
Feb 10, 2026	2.0	Reviewed threat detection rule logic; confirmed audit log fields match privacy requirements; aligned security approach with team.
Feb 11, 2026	1.5	Implemented threat detection rules for repeated failures and unusual access hours in threat_detection.py; unit-checked outputs.
Feb 12, 2026	2.0	Built /api/compliance/audit endpoint with users JOIN so actor names appear in audit trail UI; tested with Postman-style curl calls.
Feb 13, 2026	1.5	Reviewed face recognition logging: ensured access_logs stores person_id correctly; fixed duplicate-user issue by resolving by name on recognition.
Feb 14, 2026	2.0	Coordinated with team on sample registration workflow; reviewed clear_database.py and register_faces.py; tested face registration end-to-end.
Feb 17, 2026	1.5	Updated README with port configuration, FLASK_PORT=5001 requirement, and frontend folder instructions.
Feb 18, 2026	2.0	Tested full pipeline (register → recognize → log → threat → dashboard) on development machine; documented setup steps.
Feb 19, 2026	1.5	Reviewed midterm submission requirements; set up midtermSubmission branch from main with code under Implementation/ folder.
Feb 20, 2026	2.0	Wrote compliance framing for midterm report; prepared screenshots of Audit Trail and Alerts pages.
Feb 21, 2026	1.5	Aligned report structure with team; added acknowledgements and references; reviewed video demo outline.
Feb 23, 2026	5.0	Final review of midterm submission: branch, README, report structure, and video link placeholder. , Meeting with Armin

7.2 Eric Sanjo (300395898)

Date	Hours	Description of Work Done
Feb 10, 2026	2.0	Implemented API client (frontend/lib/api.ts) with typed fetch wrappers for all endpoints; added 30-second auto-refresh logic in Dashboard page component.
Feb 11, 2026	1.5	Configured Next.js proxy rewrite in next.config.ts to forward /api/* to Flask on port 5001; tested that CORS was eliminated from dev workflow.
Feb 12, 2026	2.0	Built AccessChart (hourly bar chart) and StatusDonut (donut chart) components using Recharts; wired to real API data.
Feb 13, 2026	1.5	Reviewed and fixed test_face_recognition_real.py: replaced per-face recognition counter with per-frame counter so recognition rate stays 0–100%.
Feb 14, 2026	2.0	Wrote quick_test_recognition.py and extended diagnose_recognition.py with the distinct-user encoding distance check and per-photo suspicious-photo flagging.
Feb 17, 2026	1.5	Added .jpeg support in backend sample loader (api/__init__.py) and diagnostic scripts so WhatsApp-exported images are not silently skipped.

Feb 18, 2026	2.0	Added cv2.waitKey(33) throttling to webcam test (~30 FPS instead of max speed); ran diagnostics on Eric and Reubin samples to identify overlap.
Feb 19, 2026	1.5	Updated README Getting Started section: backend from project root with FLASK_PORT=5001; frontend from frontend/ folder; port configuration explanation.
Feb 20, 2026	1.5	Created mark9 branch with correct git author configuration (user.name / user.email set to personal GitHub identity to avoid Cursor Agent co-author issue).
Feb 21, 2026	2.0	Removed Cursor Agent co-author trailers from all repo commits using git filter-branch; verified no Cursor commits remain; force-pushed all branches.
Feb 23, 2026	3.5	Final integration check: ran backend + frontend locally; confirmed stat cards, charts, alerts, and audit trail all display real data correctly. Meeting with Armin

7.3 Reubin Chatta (300394193)

Date	Hours	Description of Work Done
Feb 10, 2026	2.0	Migrated dashboard from HTML/CSS prototype to Next.js App Router structure; set up Tailwind CSS and base layout with Sidebar navigation component.
Feb 11, 2026	1.5	Built StatCard reusable component; wired to /api/stats data for entries, exits, denied events, and active threats KPIs.
Feb 12, 2026	2.0	Implemented Access Logs page (/logs) with AccessLogsTable component; added Entry/Exit and Granted/Denied/Unknown StatusBadge components.
Feb 13, 2026	1.5	Added Registered People panel to Access Logs page; implemented delete user feature with confirmation modal; wired to DELETE /api/users/<id>.
Feb 14, 2026	2.0	Implemented Alerts page (/alerts) with AlertList component; added severity filter (ALL / HIGH / CRITICAL); wired to /api/threats.
Feb 17, 2026	1.5	Implemented Audit Trail / Compliance page (/compliance) with AuditTable component; added CSV export functionality.
Feb 18, 2026	2.0	Added demo data mode: when API is unavailable or DB is empty, frontend shows seeded demo logs/alerts/audit with a visible 'Demo data' badge.
Feb 19, 2026	1.5	Tested dashboard on Windows (friend's machine); documented that backend must run first on port 5001 and frontend must be started from the frontend/ folder.
Feb 20, 2026	2.0	Created midtermSubmission branch from main; extracted full project into Implementation/ folder using git archive; committed and pushed to origin.
Feb 21, 2026	1.5	Prepared video demo outline; tested live recognition demo flow; confirmed each team member knows which section to present.
Feb 23, 2026	4.0	Final UI review and screenshot capture for midterm report; verified all pages load and display real data with backend running. Meeting with Armin

8. Closing and References

8.1 Acknowledgements

- Armin Ghauforian (Founder & CEO, Door Face Panels) — for domain expertise, hardware context (Raspberry Pi, lock, camera), requirements validation, and project guidance throughout the term.
- Priya (Instructor, CSIS 4495, Douglas College) — for guidance on research methodology, proposal requirements, midterm template, and course expectations.
- Douglas College — for access to computing resources, library services, and the Riipen partnership platform.

8.2 References

- Bradski, G., & Kaehler, A. (2008). Learning OpenCV: Computer Vision with the OpenCV Library. O'Reilly Media.
- Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), 413–422.
- Sixsmith, A., & Sixsmith, J. (2008). Ageing in place in the United Kingdom. Ageing International, 32(3), 219–235.
- Thirumurugan, P., et al. (2019). Anomaly Detection in Smart Homes Using Isolation Forest. IEEE Internet of Things Journal, 6(4), 7410–7419.
- Next.js Documentation (2024). App Router. https://nextjs.org/docs
- Flask Documentation (2024). Quickstart. https://flask.palletsprojects.com
- Privacy Commissioner of Canada. PIPEDA overview. https://www.priv.gc.ca/en/privacy-topics/privacy-laws-in-canada/the-personal-information-protection-and-electronic-documents-act-pipeda/